

Unit 1

Lesson 7

Constructor

Objective

- Concept of constructor
- Parameterized constructor
- Multiple constructors in a class

We will now discuss about ‘special member functions’ of a ‘class’ called ‘Constructors’

Constructors

Each time when you write a program you create a function, which is responsible for initializing the variable, and before the end of the program you call a function to destroy the allocated variables. To automate this feature C++ provides a concept of constructors and destructors.

C++ provides a constructor is a 'special' member function whose task is to initialize the objects of its class. It is special because its name is the same as the class name. The constructor is invoked whenever an object of its associated class is created. It is called constructor because it constructs the values of data members of the class.

A constructor is declared and defined as follows:

// class with a constructor

```
class integer
{
    int m, n;
public:
    integer(void);          // constructor declared
    .....
    .....

};
```

```
integer:: integer(void)      // constructor defined
```

```
{  
m = 0; n = 0;  
}
```

When a class contains a constructor like the one defined above, it is guaranteed that an object created by the class will be initialized automatically. For example, the declaration

```
integer int1 ;      // object int1 created
```

not only creates the object int1 of type integer but also initializes its data members m and n to zero. There is no need to write an Y, statement to invoke the constructor function (as we do with the normal member functions): If a 'normal' member function is defined for zero initialization, we would need to invoke this function for each of the objects separately. This would be very inconvenient, if there are a large number of objects.

A constructor that accepts no parameters is called the *default constructor*. The default constructor for class A is A::A(). If no such constructor is defined, then the compiler supplies a default constructor. Therefore a statement such as

```
A a;
```

invokes the default constructor of the compiler to create the object a.

Special characteristics of constructor functions :

- They should be declared in the public section.
- They are invoked automatically when the objects are created.
- They do not have return types, not even void and therefore, and they cannot return values.
- They cannot be inherited, though a derived class can call the base class constructor. . Like other C++ functions, they can have default arguments.
- Constructors cannot be virtual. (Meaning of virtual will be discussed later in Chapter 9.)
- We cannot refer to their addresses.

- An object with a constructor (or destructor) cannot be used as a member of a union. . They make 'implicit calls' to the operators new and delete when memory allocation is required.

Remember, when a constructor is declared for a class, initialization of the class objects becomes mandatory.

Parameterized Constructors

Constructor `integer()`, which we defined above, initializes the data members of all the objects to zero. However, in practice it may be necessary to initialize the various data elements of different objects with different values when they are created. C++ permits us to achieve this objective by passing arguments to the constructor function when the objects are created. The constructors that can take arguments are called *parameterized constructors*.

Constructor `integer()` may be modified to take arguments as shown below:

```
class integer {
    int m, n;
public:
    integer(int x, int y);    // parameterized constructor
};
integer :: integer(int x, int y)
{
    m = x; n = y ;
}
```

When a constructor has been parameterized, the object declaration statement such as

```
integer intl ;
```

may not work. We must pass the initial values as arguments to the constructor function when an object is declared. This can be done in two ways:

- By calling the constructor explicitly.
- By calling the constructor implicitly.

The following declaration illustrates the first method:

```
integer int1 = integer(0,100);           // explicit call
```

This statement creates an integer object int1 and passes the values 0 and 100 to it. The second is implemented as follows:

```
integer int1(0,100);                     // implicit call
```

This method, sometimes called the shorthand method, is used very often as it is shorter, looks better and is easy to implement.

Remember, when the constructor is parameterized, we must provide appropriate arguments for the constructor. Program demonstrates the passing of arguments to the constructor functions.

Ex. Of parameterized constructor

```
#include<iostream>
```

```
using namespace std;
```

```
class integer
```

```
{
```

```
int m, n;
```

```
public:
```

```
    integer(int, int);           // constructor declared
```

```
void display(void)
```

```
{
```

```
cout <<"m=" << m<<"\n";
```

```
cout <<"n=" <<n <<"\n";
```

```
}
```

```
};
```

```
integer :: integer(int x, int y)    // constructor defined
```

```
{
```

```
    m = x;
```

```
    n = y;
```

```
}
```

```
int main()
```

```
{
```

```
    integer int1(0,100);    // constructor called implicitly
```

```
    integer int2 = integer(25,75); // constructor called explicitly
```

```
    cout << "\nOBJECT1 " << "\n";
```

```
    int1.display();
```

```
    cout << "\nOBJECT2" << "\n";
```

```
    int2.display();
```

```
    return 0;
```

```
}
```

Program output:

OBJECT 1

m =0

n = 100

OBJECT2

m = 25

n = 75

The constructor functions can also be defined as **inline** functions. Example:

```
class integer
{
    int m, n;
    public:
    integer(int x, int y)    // Inline constructor
    {
        m = x; Y = n;
    }
    .....
    .....
};
```

The parameters of a constructor can be of any type except that of the class to which it belongs. For example,

```
class A
public:
    A(A);
} ;
is illegal.
```

However, a constructor can accept a *reference* to its own class as a parameter. Thus, the statement

```
Class A {
public:
    A(A&);
} ;
```

is valid. In such cases, the constructor is called the *copy constructor*.

MULTIPLE CONSTRUCTORS IN A CLASS

So far we have used two kinds of constructors. They are:

```
integer () ;    // No arguments  
integer(int, int); // Two arguments
```

In the first case, the constructor itself supplies the data values and no values are passed by the calling program. In the second case, the function call passes the appropriate values from Main(). C++ permits us to use both these constructors in the same class. For example, we could define a class as follows:

```
class integer  
{  
    int m, n;  
public :  
    integer()    // constructor 1  
    {  
        m=0; n=0;  
    }  
    integer(int a, int b) // constructor 2  
    {  
        m=a;n=b;  
    }  
    integer (integer & i) // constructor 3  
    {  
        m = i.m;  
        n = i.n;  
    }  
}
```

This declares three constructors for an integer object. The first constructor receives no arguments, the second receives two integer arguments and the third receives one integer object as an argument.

For example, the declaration

```
integer I1;
```

would automatically invoke the first constructor and set both m and n of I1 to zero. The statement

```
integer I2(20,40);
```

would call the second constructor which will initialize the data members m and n of I2 to 20 and 40 respectively. Finally, the statement

```
integer I3(12);
```

would invoke the third constructor which copies the values of I2 into I3. In other words, it *sets* the value of every data element of I3 to the value of the corresponding data element of I2.

As mentioned earlier, such a constructor is called the *copy constructor*. We learned in Chapter 4 that the process of sharing the same name by two or more functions is referred to as function overloading. Similarly, when more than one constructor function is defined in a class, we say that the constructor is overloaded.

I

```
#include<iostream>
```

```
using namespace std;
```

```
class complex
```

```
{
```

```
float x,y;
```

```
public :
```

```
complex() // constructor no arg
```

```
{
```

```
}
```

```
complex(float a) // constructor – one argument
```

```
{ x = y = a;
```

```
}
```

```
complex( float real, float imag) // constructor – two arg
```



```

{
    x = real;
    y = imag;
}

friend complex sum(complex ,complex);
friend void show(complex);
};

complex sum(complex c1,complex c2) // friend
{
    complex c3;
    c3.x=c1.x+c2.x;
    c3.y=c1.y+c2.y;
    return(c3);
}

void show(complex c) // friend
{
    cout <<c.x<<"+"j"<<c.y<<"\n";
}

int main()
{
    complex A(2.7,3.5); //define & initialize
    complex B(1.6);    //define and initialize
    complex C; //define

    C=sum(A,B);
    Cout << "A="; show(A); //sum() is a friend

    Cout << "B="; show(B); //sum() is a friend

    cout << "C="; show(C); //sum() is a friend

```

//another way to give initial values (second method)

```
complex P,Q,R;    // define P, Q ,R
p= complex(2.5,3.9); // initialize P
Q= complex(1.6,2.5); // initialize Q
R= sum(P,Q);
```

```
Cout <<"\n";
Cout <<"P = "; show(P);
Cout <<"Q = "; show(Q);
```

```
Cout <<"R = "; show(R);
Return 0;
}
```

output will be

```
A=2.7+J3.5
B=1.6+J1.6
C=4.3+J5.1
```

```
P=2.5+J3.9
Q=1.6+J2.5
R=4.1+J6.4
```

There are three constructors in the class complex. The first constructor, which takes no arguments, is used to create objects which are not initialized; the second, which takes one argument, is used to create objects and initialize them; and the third, which takes two arguments, is also used to create objects and initialize them to specific values. Note that the second method

Summary

- C++ provides a constructor is a 'special' member function whose task is to initialize the objects of its class. It is special because its name is the same as the class name.
- The constructor is invoked whenever an object of its associated class is created. It is called constructor because it constructs the values of data members of the class.
- A constructor that accepts no parameters is called the default constructor. The default constructor for class A is A::A().

Questions

1. Define constructor concept along with example?
2. What is parameterised constructor ? explain with example.?
3. how do we invoke a constructor function?
4. Can we have more than one constructor in a class? If yes, explain the need for such situation.